

Week 10

2. RDBMS: SQL

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Wednesday, March 11th, 2026 | 14:30 – 15:00 | ER 1118



University of Windsor

Agenda

- Recap on Database Systems
- The Relational Database Model
- The Structured Query Language (SQL)
- INSERT – DELETE – UPDATE
- Problems in File Environments
- About Databases
- Types of Databases
- RDBMS
- Wrap-up



Part I Recap





Recap



University of Windsor

Database Systems

- Data vs. Information
 - Collection → Storage → Access/Manipulation/visualization → Analysis/Mining
 - DBMS contains information about a particular enterprise
 - Collection of interrelated data
 - Set of programs to access the data
 - An environment that is both *convenient* and *efficient* to use (storage, security, etc.)
- Database systems are used to manage collections of data that are:
 - Highly valuable
 - Relatively large
 - Accessed by multiple users and applications, often at the same time.
- A modern database system is a complex software system whose task is to manage a large, complex collection of data.
- Databases touch all aspects of our lives



Part II

The Relational Database Model





About Relations



Equivalent Sets of Terms

Table	Row	Column
File	Record	Field
Relation	Tuple	Attribute

Characteristics of a Relation

1. Rows contain data about an entity
2. Columns contain data about attributes of the entity
3. Cells of the table hold a single value
4. All entries in a column are of the same kind
5. Each column has a unique name
6. The order of the columns is unimportant
7. The order of the rows is unimportant
8. No two rows may hold identical sets of data values



Characteristics of a Relation

1. Rows contain data about an entity
2. Columns contain data about attributes of the entity
3. Cells of the table hold a single value
4. All entries in a column are of the same kind
5. Each column has a unique name
6. The order of the columns is unimportant
7. The order of the rows is unimportant
8. No two rows may hold identical sets of data values

STUDENT (Relation 1)

StudentID	Name	Age	Program	Course1	Course2
1	John Doe	20	Analytics	Programming	Statistics
2	Sally Smith	22	Business	Microeconomics	Macroeconomics
3	Peter Parker	twenty	Science	Chemistry	Biology
4	Clark Kent	23	Journalism	English, Storytelling, Writing	

STUDENT (Relation 2)

StudentID	Name	Age	Program	Course	Course
1	John Doe	20	Analytics	Programming	Statistics
2	Sally Smith	22	Business	Microeconomics	Macroeconomics
3	Peter Parker	twenty	Science	Chemistry	Biology
4	Clark Kent	23	Journalism	English, Storytelling, Writing	
1	John Doe	20	Analytics	Programming	Statistics



Example of the Instructor Relation

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

attributes
(or columns)

tuples
(or rows)

Example of the Courses Relation

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

attributes
(or columns)

tuples
(or rows)

Example of the Prerequisites Relation

- A “Join” Table

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101



Representing Relations

- TABLE_NAME (Field_1, Field_2, ... , Field_n)
- If a primary key is known, underline it:
 - TABLE_NAME (Field_1, Field_2, ... , Field_n)



Attributes

- The set of allowed values for each attribute is called the **domain** of the attribute
 - E.g., valid dates for a date of birth
- Atomic (indivisible) vs. Non-atomic domain
 - Domain is ideally atomic
 - There are exceptions, e.g., name can be both atomic or non-atomic (first & last)
 - Date of birth can be both atomic or non-atomic
- The special value ***null*** is a member of every domain. Indicated that the value is “unknown”
 - Special care is required
 - The null value causes complications in the definition of many operations

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



Keys

- A **superkey** is a collection of attribute values that can uniquely identify each entity or a row in a table.
{ID} and *{ID,name}* are both superkeys of *instructor*.
- A superkey is a **candidate key** if no proper subset is a superkey
{ID} is a candidate key
- One of the candidate keys is selected to be the **primary key**

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



○ A Relational Database Example



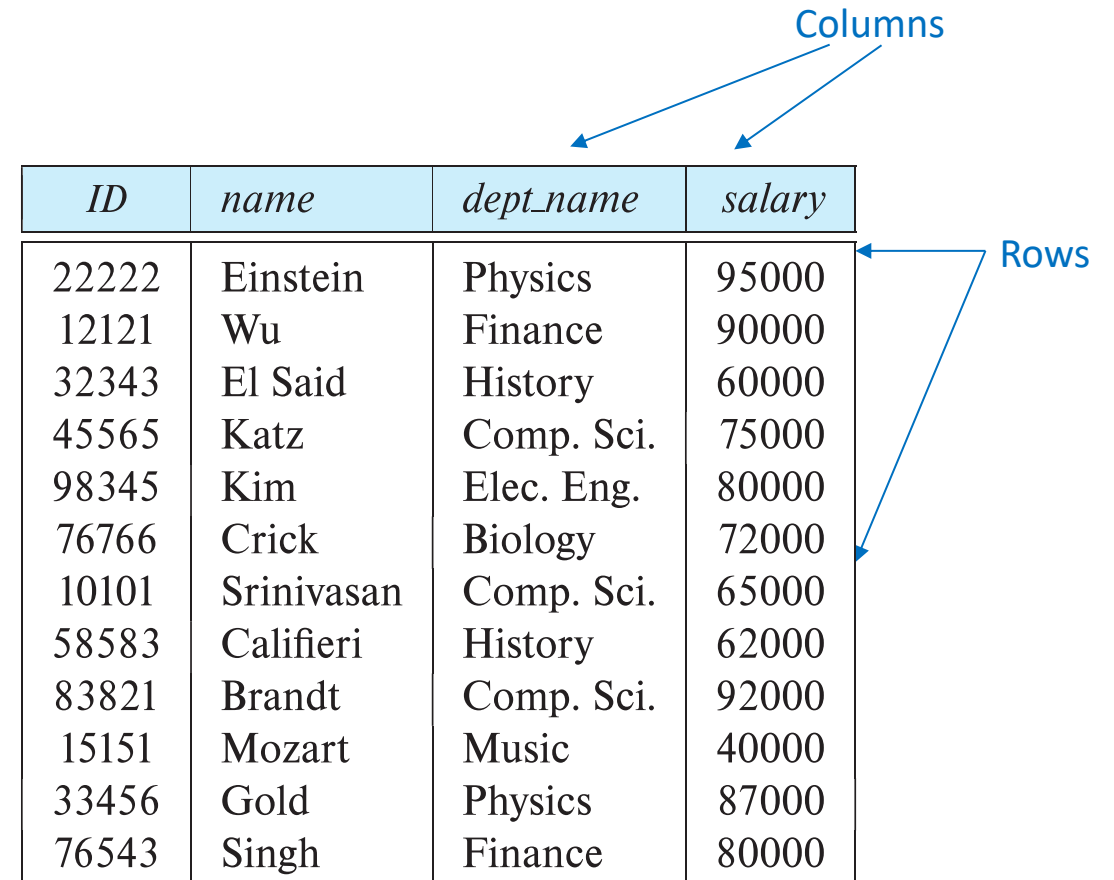
University Database Example

- Let's consider a university database to illustrate the concepts
- Data consists of information about:
 - Students
 - Instructors
 - Classes
- Application program examples:
 - Add new students, instructors, and courses
 - Register students for courses, and generate class rosters
 - Assign grades to students, compute grade point averages (GPA) and generate transcripts



Relational Model

- Other Name: Record-based model.
- All the data is stored in various tables.
- Example of tabular data in the Relational Model.



The diagram illustrates a table structure. Two arrows labeled "Columns" point to the header row, which contains the column names: *ID*, *name*, *dept_name*, and *salary*. Another arrow labeled "Rows" points to the data rows, which contain individual records. The table is as follows:

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Levels of Abstraction

- **Physical level:** Describes how a record (e.g., instructor) is stored.
 - Bytes, memory blocks, etc.
- **Logical level:** Describes data stored in database, and the relationships among the data. (used by database administrators)

- Example from programming world

```
type instructor = record
```

```
    ID:          string;
```

```
    name:        string;
```

```
    dept_name:  string;
```

```
    salary:     integer;
```

```
end;
```

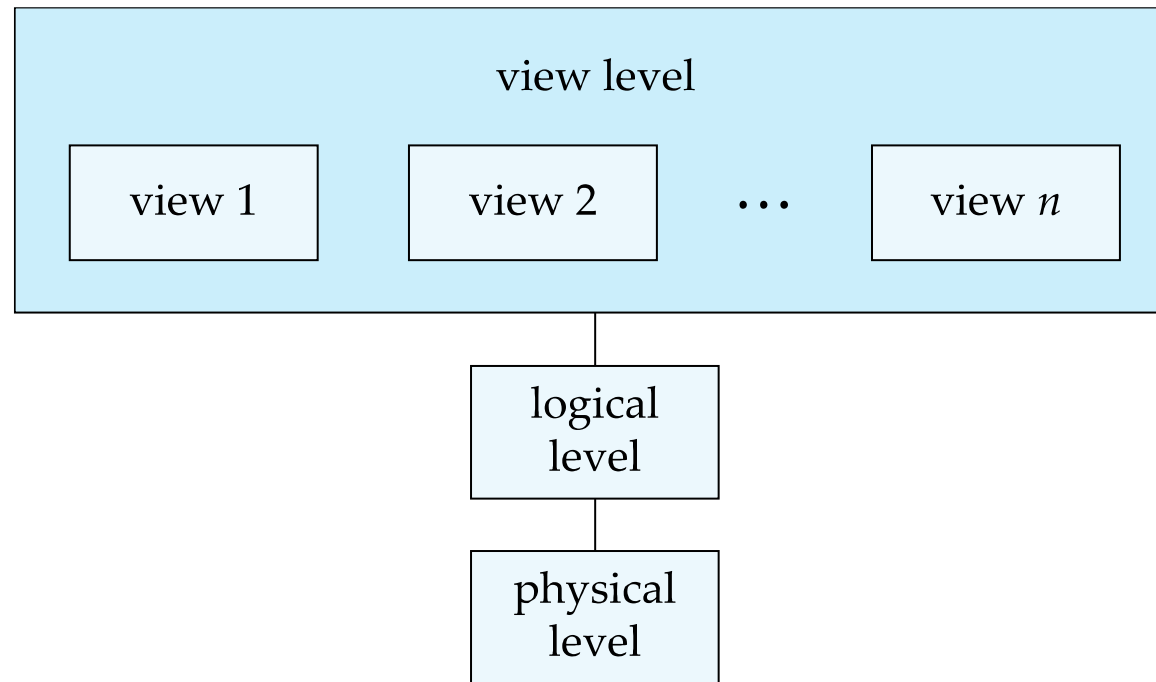
- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

- Fulfills users' needs and put constraints on who can see what.



View of Data

- An architecture for a database system



Instances and Schema

- Similar to types and variables in programming languages
- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
 - Good schema vs. bad schema
- **Physical schema** – the overall physical structure of the database
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable



Part III

The Structured Query Language (SQL)



SQL Basics



SQL History

- Invented by Don Chamberlin and Ray Boyce at IBM, initially called SEQUEL
- The standard programming language for creating, updating and retrieving information that is stored in databases
- It is:
 - A ***programming language*** – a formal language in which to write programs to create, modify, and query databases.
 - Defined by rules of ***syntax*** (determine the words and symbols you can use and how they are combined)



SQL Characteristics

- It is ***Declarative*** – you describe what you want, and the database will determine how to do it
- It is ***Interactive*** – you issue SQL commands directly to your Database Management System (Access, SQL Server, MySQL, etc.) and it displays the results
- OR it is ***Embedded*** – you can embed the SQL statements in a scripting language (PHP, Java, etc.)

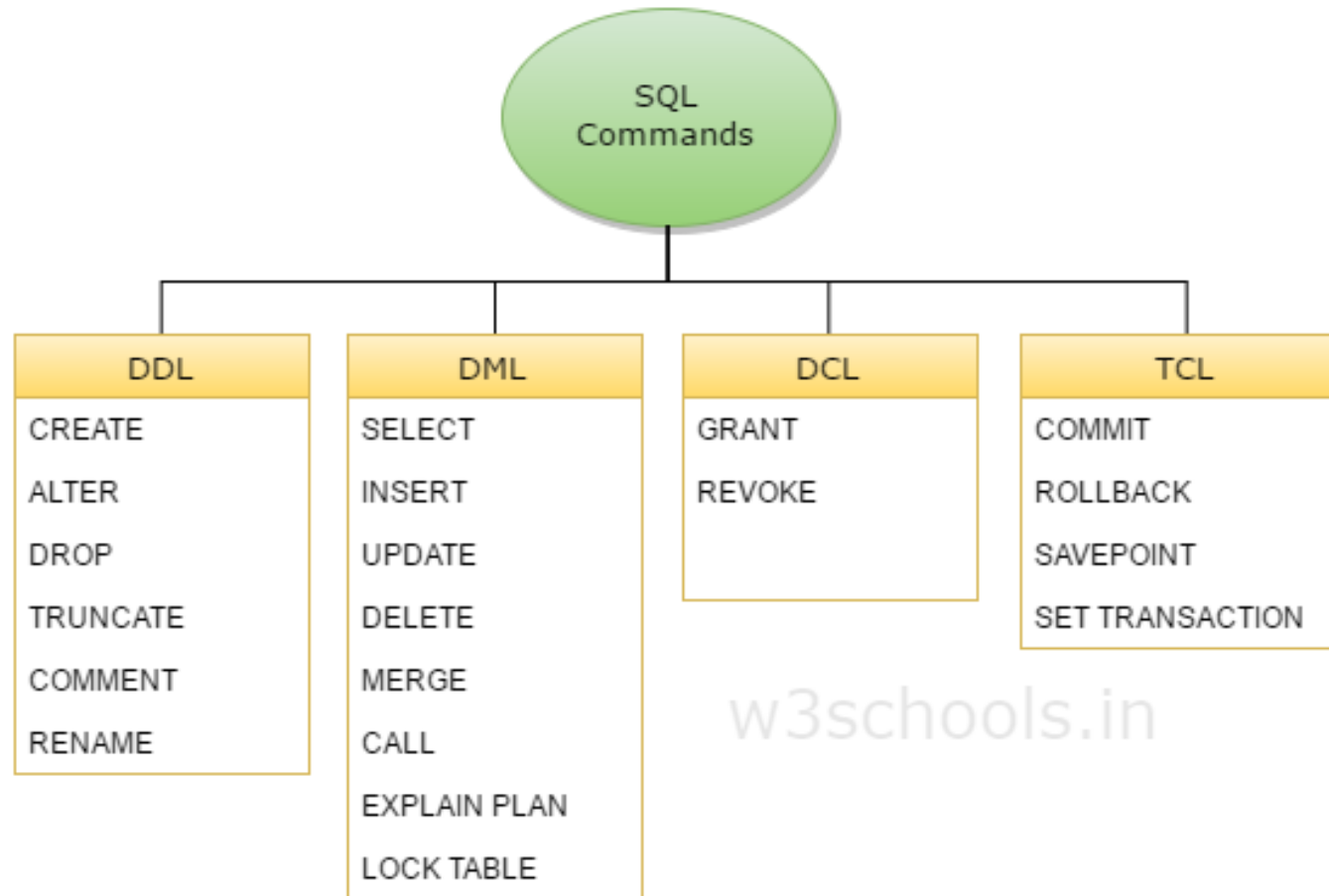


SQL

- It is ***Standardized*** – no one owns it, but it is a standard that is defined by an international standards working group (American National Standards Institute – ANSI) ...
 - ... but not RDBMSs are fully ANSI compatible
- It is commonly referred to as **Structured Query Language (SQL)**



SQL Command Groups



SQL Categories

- **Data Definition Language (DDL)** – creating tables, relationships, and other structures
- **Data Manipulation Language (DML)** – querying, inserting, modifying, and deleting data
- **SQL/Persistent Stored Modules (SQL/PSM)** – extend SQL by adding procedural programming capabilities including flow-of-control



SQL Syntax



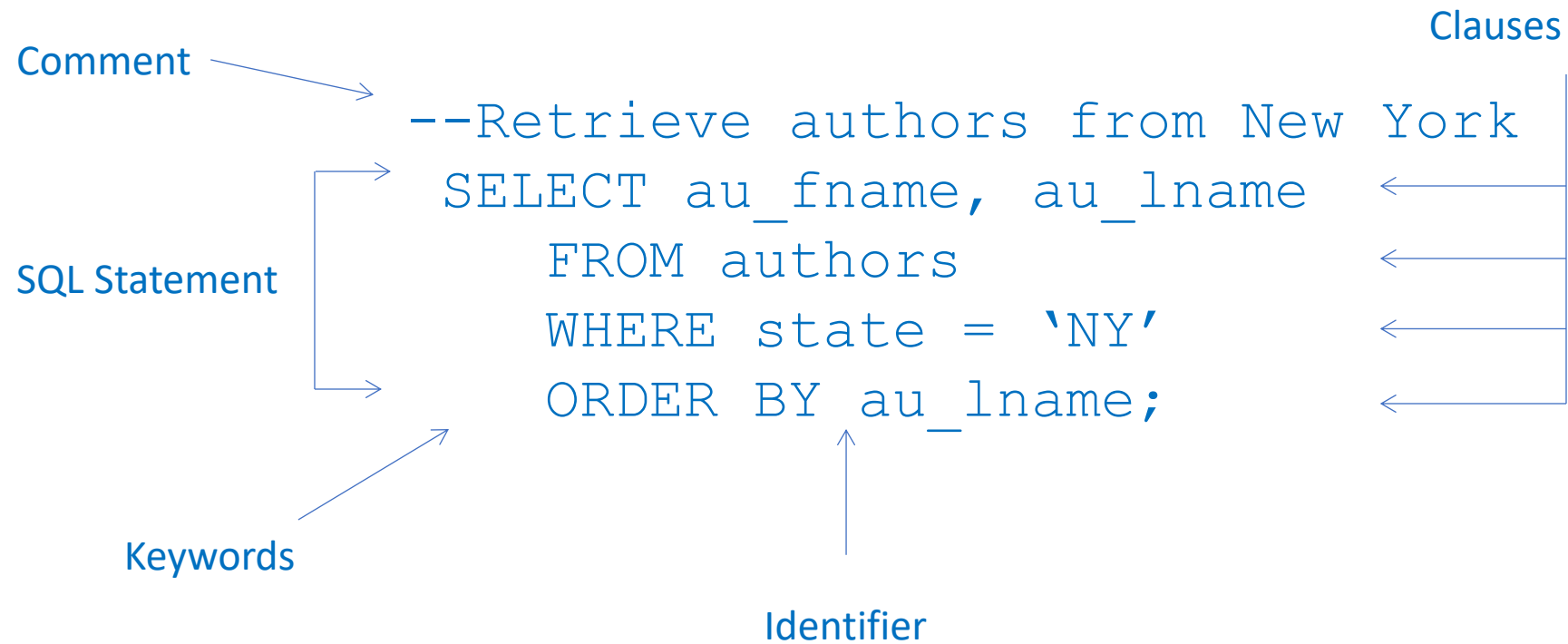
SQL Syntax

- **Keywords** – words that SQL reserves because they have special meaning in the language.
 - SELECT, CREATE, TABLE
- **Identifiers** – words that you use to name objects, columns, aliases, indexes and views
 - authors, titles, books,
- **Terminating semicolon ;** – ends with an SQL statement (ACCESS and SQL server do not require)



● SQL Syntax

○ Example of SQL Syntax



SQL Syntax

- SQL is a free-form language whose statements can:
 - Be in uppercase or lowercase – `SELECT`, `Select`, `select`, `SeLeCt` are considered to be identical
 - Continue on the next line
 - Be on the same line as other statements
 - Start in any column



Equivalent Statements

- The statements below are equivalent

```
SELECT au_fname,  
       au_lname  
FROM authors  
WHERE state = 'NY'  
ORDER BY au_lname;
```

```
Select au_fname  
       AU_LNAME  
FROM  
Authors WhERe state  
= 'NY' order  
by  
AU_lname  
;
```

SQL Syntax

- **Comment** – text that explains your program
- **SQL statement** – a valid combination of **tokens** introduced by a **keyword**
- **Tokens** – the indivisible particles of the SQL language. They include keywords, identifiers, operators, literals (constants) and punctuation symbols
- **Clauses** – a fragment of an SQL statement that's introduced by a keyword, is required or optional and must be given in a particular order (SELECT, FROM, WHERE and ORDER BY introduce the four clauses in the example)



Syntax Conventions

- Each SQL statement begins on a new line
- The indentation level is two spaces
- Each clause begins on a new, indented line
- SQL is case insensitive: myname, MyName, and MYNAME are considered to be identical identifiers
- Generally, use uppercase for SQL keywords and lowercase for user-defined values.

```
SELECT au_fname, au_lname  
      FROM authors  
      ORDER BY au_lname
```



You WILL follow these conventions



● Naming recommendations (User-defined Values)

- • Use lowercase letters
- `names_with_underscores` are easier to read than `nameswithoutthem`
- Use the same conventions/abbreviations throughout the database



Table/Column Names

- You can use a quoted identifier (delimited identifier) to break some of the rules – this involves surrounding the name with double quotes or square brackets

`[customer addresses]`
`"customer addresses"`

It is not recommended

SQL DDL & DML



SQL

- **DDL** – Provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- **DML** – Provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.



○ SQL – DDL



Data Definition Language (DDL)

- Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a ***data dictionary***
- Data dictionary contains metadata (i.e., data about data)
 - Database schema
 - Integrity constraints
 - Primary key (ID uniquely identifies instructors)
 - Referential integrity: a student registering for a course must exist
 - Authorization
 - Who can access what



Create Table Construct

- An SQL relation is defined using the **create table** command:

create table *r*

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 $(\text{integrity-constraint}_1),$

$\dots,$
 $(\text{integrity-constraint}_k))$

- *r* is the name of the relation
 - each A_i is an attribute name in the schema *R* of relation *r*
 - D_i is the data type of values in the domain of attribute A_i
- Example:

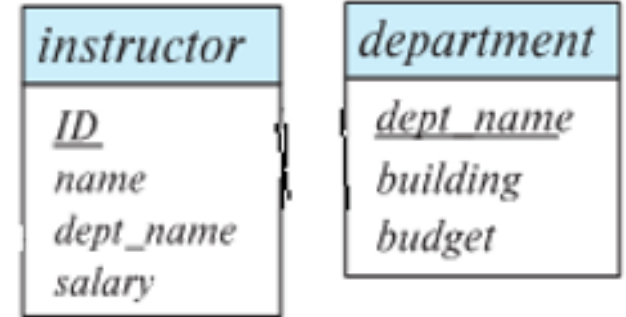
```
create table instructor (  
    ID          char(5),  
    name        varchar(20),  
    dept_name   varchar(20),  
    salary      numeric(8,2))
```



Integrity Constraints

- Types of integrity constraints
 - primary key** ($A_1, \dots, A_n$)
 - foreign key** ($A_m, \dots, A_n$) references r
 - not null**
- SQL prevents any update to the database that violates an integrity constraint.
- Example:

```
create table instructor (  
    ID          varchar(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    salary      numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references  
department);
```



Another Example

- **create table student** (
 ID *varchar*(5),
 name *varchar*(20) not null,
 dept_name *varchar*(20),
 tot_cred *numeric*(3,0),
 primary key (ID),
 foreign key (dept_name) **references** department);
- **create table takes** (
 ID *varchar*(5),
 course_id *varchar*(8),
 sec_id *varchar*(8),
 semester *varchar*(6),
 year *numeric*(4,0),
 grade *varchar*(2),
 primary key (ID, course_id, sec_id, semester, year) ,
 foreign key (ID) **references** student,
 foreign key (course_id, sec_id, semester, year)
 references section);

student
<u>ID</u>
name
dept_name
tot_cred

department
<u>dept_name</u>
building
budget

takes
<u>ID</u>
<u>course_id</u>
<u>sec_id</u>
<u>semester</u>
<u>year</u>
grade

section
<u>course_id</u>
<u>sec_id</u>
<u>semester</u>
<u>year</u>
building
room_number
time_slot_id

○ SQL – DML



Data Manipulation Language

- Language for accessing and updating the data organized by the appropriate data model
 - DML also known as query language (technically incorrect)
- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify ***what*** data are needed and ***how*** to get those data.
 - **Declarative DML** -- require a user to specify ***what*** data are needed without specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as non-procedural DMLs



SQL Query Language

- A query takes as input several tables (possibly only one) and returns a table.

- Example to find all instructors in Comp. Sci. dept

```
select name  
  from instructor  
  where dept_name = 'Comp. Sci.'
```

- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database





SELECT



University of Windsor

SELECT Clause

- A typical SQL query has the form:

```
select  $A_1, A_2, \dots, A_n$   
      from  $r_1, r_2, \dots, r_m$   
      where  $P$ 
```

- A_i represents an attribute
- R_i represents a relation
- P is a predicate.
- The result of an SQL query is:
 - A relation.

SELECT Clause (cont.)

- Example: find the names of all instructors:
select *name*
from *instructor*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

<i>name</i>
Srinivasan
Wu
Mozart
Einstein
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim

SELECT Clause (cont.)

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.

- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name  
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name  
from instructor
```

```
select dept_name  
from instructor
```

dept_name

Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.



SELECT Clause (cont.)

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.

- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name  
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name  
from instructor
```

```
select dept_name  
from instructor
```

An asterisk in the select clause
denotes “all attributes”

```
select * from instructor
```

dept_name

Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.





WHERE



University of Windsor

The WHERE Clause

- The **WHERE** clause specifies conditions that the result must satisfy:
 - Corresponds to the selection predicate of the relational algebra.



The WHERE Clause

- How to find all instructors in Comp. Sci department?

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



The WHERE Clause

- How to find all instructors in Comp. Sci department?

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

To find all instructors in Comp. Sci. dept
select *name*
from *instructor*
where *dept_name* = 'Comp. Sci.'



The WHERE Clause

- How to find all instructors in Comp. Sci department?

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

To find all instructors in Comp. Sci. department:

```
select name  
  from instructor  
    where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and**, **or**, and **not**
- The operands of the logical connectives can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**.



Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



○ AGGREGATE Functions



AGGREGATE Functions

- Find the average salary of instructors in the Computer Science department :

```
select avg (salary)  
from instructor  
where dept_name= 'Comp. Sci.';
```

- Should the duplicates be allowed?
 - Yes for Average

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

AGGREGATE Functions

- Find the total number of instructors who teach a course in the Spring 2018 semester

```
select count (ID)  
from teaches  
where semester = 'Spring'  
and year = 2018;
```

<i>teaches</i>
<u>ID</u>
<u>course_id</u>
<u>sec_id</u>
<u>semester</u>
<u>year</u>

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



AGGREGATE Functions

- Find the total number of instructors who teach a course in the Spring 2018 semester

```
select count (ID)  
from teaches  
where semester = 'Spring'  
and year = 2018;
```

Is there anything wrong with the above query?

Correct:

```
select count (distinct ID)  
from teaches  
where semester = 'Spring'  
and year = 2018;
```

teaches				
<u>ID</u>				
<u>course_id</u>				
<u>sec_id</u>				
<u>semester</u>				
<u>year</u>				

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



Part XI

Modifying the Database:

INSERT – DELETE – UPDATE



Operations on Tables

- **Insert**

```
insert into instructor  
values ('10211', 'Smith', 'Biology', 66000);
```

- **Delete**

- Remove all tuples from the *student* relation
`delete from student` *(the relation instance remains with 0 tuples)*

- **Update Table**

- **Drop Table**

```
drop table r (the relation does not exist anymore)
```

- **Alter Table**

- `alter table r add A D`
 - where *A* is the name of the attribute to be added to relation *r* and *D* is the domain of *A*.
 - All existing tuples in the relation are assigned *null* as the value for the new attribute.
- `alter table r drop A`
 - where *A* is the name of an attribute of relation *r*
 - Dropping of attributes not supported by many databases.





INSERT



University of Windsor

INSERT

- **Syntax:**

```
INSERT INTO TableName (Column1, Column2,...)
VALUES (Value1, Value2, ...)
```

- Inserts a new row into the table

- Also possible to insert bulk set of rows:

```
INSERT INTO Table1 (Column1, Column2,...)
SELECT (ColumnA, ColumnB, ...)
FROM Table2;
```

INSERT

- Add a new tuple to *course*

```
insert into course  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- or equivalently

```
insert into course (course_id, title, dept_name, credits)  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- Add a new tuple to *student* with *tot_creds* set to null

```
insert into student  
values ('3003', 'Green', 'Finance', null);
```



INSERT

- Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000.

```
insert into instructor
  select ID, name, dept_name, 18000
 from   student
 where  dept_name = 'Music' and total_cred > 144;
```

- The **select from where** statement is evaluated fully before any of its results are inserted into the relation.
- Most relational database products have special “bulk loader” utilities to insert a large set of tuples into a relation. These utilities allow data to be read from formatted text files, and they can execute much faster than an equivalent sequence of insert statements.





DELETE



University of Windsor

● DELETE

○ DELETE

FROM

WHERE

TableName

Column1 = Value1;



DELETE

- Delete all instructors (does not delete the schema)
`delete from instructor`
- Delete all instructors from the Finance department
`delete from instructor
where dept_name= 'Finance';`
- Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.

```
delete from instructor  
where dept name in (select dept_name  
                    from department  
                    where building = 'Watson');
```



DELETE

- Delete all instructors whose salary is less than the average salary of instructors:

```
delete from instructor  
where salary < (select avg (salary)  
from instructor);
```



DELETE

- Delete all instructors whose salary is less than the average salary of instructors

```
delete from instructor  
  where salary < (select avg (salary)  
                    from instructor);
```

- Problem: as we delete tuples from *instructor*, the average salary changes
- Solution used in SQL:
 1. First, compute **avg** (*salary*) and find all tuples to delete
 2. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples)



● In-class Activity: DELETE in Northwinds

- • Use the online version of the Northwinds database ([SQL Online IDE – Fast SQL Editor | SQL Compiler](#))
- Remove Robert Zare from the Employees table.
- Delete any order from the Orders table where the Shipped Date field is empty.
 - Hint: Empty means ... NULL



DELETE in Northwinds (Solution)

- Remove Robert Zare from the Employees table.

```
DELETE      *  
FROM        Employees  
WHERE       [Last Name] = "Zare";
```

- Relationship with other tables should prevent the delete.



DELETE in Northwinds (Solution)

- Delete any order from the Orders table where the Shipped Date field is empty:

```
DELETE *  
FROM Orders  
WHERE [Shipped Date] IS NULL
```





UPDATE



University of Windsor

UPDATE

- **Syntax:**

```
UPDATE  TableName  
      SET  Column1 = Value1, Column2 = Value2, ...  
      WHERE ColumnX = ValueX;
```

- Updates a value or values in a row in the table



UPDATE

- Give a 5% salary raise to all instructors

update *instructor*

set *salary* = *salary* * 1.05

- Give a 5% salary raise to those instructors who earn less than 70000

update *instructor*

set *salary* = *salary* * 1.05

where *salary* < 70000;



● In-class Activity: UPDATE in Northwinds

- • Using the Customers table, update Thomas Axen's business phone number to (555)555-1235
- Update the Northwind Traders Olive Oil quantity per unit sold to 25 boxes. Use the Products table



● In-class Activity: UPDATE in Northwinds (solution)

- • Using the Customers table, update Thomas Axen's business phone number to (555)555-1235:

```
UPDATE Customers  
SET [Business Phone] = "(555) 555-1235"  
WHERE [Last Name] = "Axen";
```



In-class Activity: UPDATE in Northwinds (solution)

- Use the online version of the Northwinds database ([SQL Online IDE – Fast SQL Editor | SQL Compiler](#))
- Update the Northwind Traders Olive Oil quantity per unit sold to 25 boxes. Use the Products table:

```
UPDATE      Products
SET         [Quantity Per Unit] = "25 boxes"
WHERE      [Product Name] LIKE "*olive oil";
```



Part XI

Wrap-up – What's Next



Summary



Summary

In this lecture:

- We were introduced to the Structured Query Language (SQL), the programming language used to query databases supporting the Relational Database Model (RDBMS).
- We worked on the main Data Manipulation Language (DML) commands of SQL:
 - SELECT
 - FROM
 - WHERE
- Performed in-class exercises.



Next Lecture



University of Windsor

Next Lecture

- Hands-on to Database Programming in Java
 - MySQL (open source?)
 - Apache Derby (open source)



Part XII

Training and resources





Tools



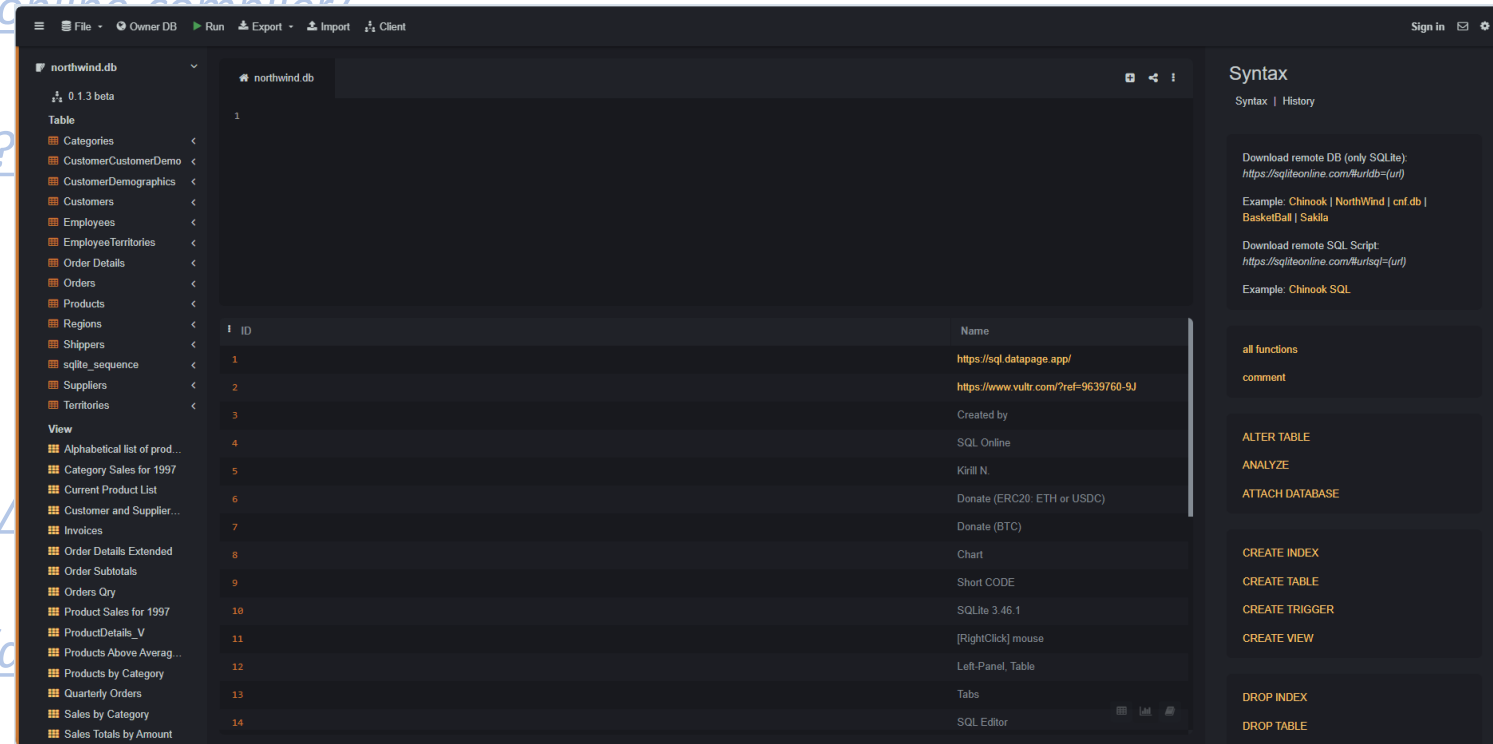
Tools

- <https://www.essentialsql.com/sql-tutorial-for-sql-server/>
- https://www.w3schools.com/sql/sql_editor.asp
- <https://www.programiz.com/sql/online-compiler/>
- <https://sqliteonline.com/>
- <https://livesql.oracle.com/apex/f?p=590:1000>
- Code editors:
 - Visual Studio Code / VS Codium
 - Eclipse IDE
 - Notepad++, etc.
- DBeaver: <https://dbeaver.io/>
- SqlDBM: <https://app.sqldbm.com/>
- <https://dbmstools.com/>
- Oracle: <https://www.oracle.com/database/sqldeveloper/technologies/sql-data-modeler/>



Tools

- <https://www.essentialsql.com/sql-tutorial-for-sql-server/>
- https://www.w3schools.com/sql/sql_editor.asp
- <https://www.programiz.com/sql/online-compiler/>
- <https://sqliteonline.com/>
- https://livesql.oracle.com/apex/f?_af=1
- *Visual Studio Code*
- *Notepad++*
- *DBeaver*: <https://dbeaver.io/>
- *SqlDBM*: <https://app.sqldbm.com/>
- <https://dbmstools.com/>
- *Oracle*: <https://www.oracle.com/>



○ Training resources



Training Resources

- Datacamp: <https://app.datacamp.com/learn/courses/introduction-to-sql>
- <https://www.essentialsql.com/>
- <https://www.w3schools.com/sql/>
- <https://www.khanacademy.org/computing/computer-programming/sql/sql-basics/v/welcome-to-sql>
- <https://www.learnsqlonline.org/>
- <https://sqlbolt.com/>
- https://sqlzoo.net/wiki/SQL_Tutorial
- <https://www.codecademy.com/resources/docs/sql>
- <https://www.freecodecamp.org/>
- ...



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor